

Wizard's Academy

Diksha Chottani

December 6, 2025

Declaration of Originality

I, Diksha Chottani, hereby declare that the work presented in this project report, including all programming code (C++ and GLSL shaders) and the written assignment, is entirely my own original work.

I confirm that:

- I have not collaborated with any other individual on the programming or documentation tasks.
- I have not used any source code found online, except for standard open-source libraries (GLFW, GLAD/GLEW, Assimp, GLM) which are explicitly cited in the References section.

Name (Typed Signature): Diksha Chottani **Date:** December 6, 2025

Contents

Declaration of Originality	2
1 Introduction	4
1.1 Project Overview and Goal	4
1.2 Theme and Core Concept	4
1.3 Report Structure	4
2 Ecosystem Design and Initial Proposal	5
2.1 Key Ecosystem Elements	5
2.2 Scope Revision: Dynamic Lighting and Floating Island Environment	5
2.3 Scene Layout and Model Inspiration	5
2.4 Advanced and Custom Techniques Rationale	5
3 Core Technical Implementation	6
3.1 Basic Scene Setup and Object Loading	6
3.2 User Camera Control	7
3.3 Hierarchical Animation	8
4 Lighting and Shading Implementation	9
4.1 Dynamic time-of-Day System (Advanced Feature	9
4.2 Phong illumination and Light Sources	10
4.3 Texture Mapping	11
5 Advance Rendering and Final Polish	11
5.1 Dynamic Magical Effects	11
5.2 Complex Time-Based Transformation: Floating Wizard	12
5.2.1 Overall Aesthetic and Immersive Quality:	13
6 Conclusion and Future work	13
6.1 Project Summary and Achieved Objectives	13
6.2 Challenges and Learning Outcomes	14
7 References and Appendices	14
7.1 Source Code Repository	14
7.2 Technical References	14
7.3 Asset References	15
7.4 Appendix A: Demonstration Video	15

1 Introduction

1.1 Project Overview and Goal

The main goal of this project was to build a Fantasy Ecosystem Simulation using the required shader-based Open-GL framework. This involved implementing a complete graphics pipeline, covering essential tasks like implementing a complete graphics pipeline, covering essential tasks like loading 3D models, creating interactive camera controls, and developing advanced shading and animation techniques. The selected theme, The Wizard's Academy, was used as the context to demonstrate these technical skills, with a specific focus on stylized visual effects and complex rendering.

1.2 Theme and Core Concept

The simulation shows a training session taking place in the courtyard of a secluded Wizard's Academy. Crucially, in the final implementation, the academy is situated on a fantastical floating island. This creative change was inspired by reading a fantasy book during the project, where empires and cities were built on floating island, providing a compelling new structure for the simulation. The overall style is an animated fantasy setting, focusing on a mood of mystical but focused learning, rendered with a dynamic time-of-day cycle.

- **Scene Focus:** Showing students practicing magic spells under the instructions of the Head Wizard.
- **Key Visual Element:** The environment features a unique dynamic time-of-day system. This system is implemented while maintaining the visual constraints of an always blue sky to represent the magical realm, using the shifting shadow intensity, color temperature, and object luminosity to clear differentiate the time of day.
- **Magical Effects:** A core requirement is still the creation of glowing magical effects. This required advanced blending and custom shaders to ensure the spells look bright and luminous across all lighting conditions. from bright sunlight to deep shadows.

1.3 Report Structure

This report is organized to follow the project's development across the three main deliverables:

- **Section 2: Ecosystem Design(Deliverable 1):** Covers the initial design, key elements, and documents the crucial changes in scope to a floating island and the dynamic time-of-day cycle.
- **Section 3: Core Implementation (Deliverable 2):** Details the foundational technical work, including 3D model loading, user camera control, and the implementation of hierarchical animation.
- **Section 4 and 5: Advanced Feature and Polish (Deliverable 3):** Details the final, complex technical implementation, such as Phong illumination, texture mapping, and the specialized advanced features crucial to final aesthetic.

2 Ecosystem Design and Initial Proposal

2.1 Key Ecosystem Elements

The environment is built around the magical training setting:

- **Environment:** The setting is a large, open training courtyard surrounded by classical fantasy architecture. The central wizard statue serves as a focal point, representing the academy's history.
- **Characters:** The simulation features the static, commanding Head Wizard and the dynamic Student Wizards. The students' synchronized spell casting movements represent the project's thematic core.

2.2 Scope Revision: Dynamic Lighting and Floating Island Environment

The project scope was strategically changed during implementation to introduce unique technical challenges and the fantasy settings. This deviation from the original night-time, valley-based proposal was directly motivated by creative inspiration from fantasy literature featuring flying empires, which offered more exciting possibilities for terrain and atmosphere.

- **Setting Change (Ground to Air):** The academy is now situated on a floating island, rather than in a secluded valley. This modification required implementing a detailed skybox and careful management of the terrain's edged to simulate like other floating islands.
- **Lighting Change (Static Night to Dynamic Cycle):** The static night-time setting was replaced with a dynamic time-of-day cycle. This system controls the position, color, and intensity of the primary directional light and ambient light. Critically, the system is designed to simulate the sky of a magical realm, meaning the sky color remains predominantly blue throughout the cycle, with the time of day being communicated entirely through changing shadows, scene color temperature, and local object luminosity.

2.3 Scene Layout and Model Inspiration

The core layout remains a central training circle. Inspiration was drawn from visual references to establish the animated fantasy aesthetic, incorporating elements of old stone architecture and mystical, ethereal lighting.

- A placeholder is used to represent the initial concept sketch illustrating the planned circular courtyard layout, relative positions of the statue, Head Wizard, and student group.

2.4 Advanced and Custom Techniques Rationale

The final implementation of the project's visual identity relies on adapting the planned advanced techniques to function within the unique constraints of the floating island and the dynamic lighting cycle.

- **Dynamic Time-of-Day System:** The core aesthetic achievement is the dynamic lighting cycle, which smoothly transitions the scene through Daylight, Sunset, and Night. This system continuously calculates the sun's position and uses this data to update the color and intensity of the scene's primary directional light. This is a critical technical choice, as the time of day is conveyed primarily through the changing shadow angles and color temperature, since the magical realm's sky color largely constant (blueish).

- **Custom Particle System for Magical Spells:** The glowing spells required a dedicated, efficient system. This was achieved using a custom particle rendering method based on billboarded 2D geometry.
 - **Motion:** Each spell effect generates particles that follow a pre-calculated path combining circular orbits and vertical oscillation. This creates the visual effect of controlled, swirling energy being channeled by the wizards.
 - **Billboard Technique:** The 2D particle geometry is always oriented to face the camera, ensuring the visual element appears consistently visible and voluminous regardless of the user's viewpoint.
- **Additive blending for Luminosity:** To ensure the spells actually appear to glow and emit light against the bright environment, the crucial rendering technique of Additive Blending was utilized. By setting the blending function to add the spell's color to the background color, the magical effects maintain their visual intensity across all lighting conditions, from bright noon to deep night.
- **Volumetric Fog:** While the original plan focused on night-time fog, this feature was adapted to function across the full dynamic cycle. The implementation of volumetric effects must now be optimized to create sharp light shafts under the directional daylight and transition smoothly to soft, ethereal mist during night and sunset, accurately reflecting the light source's current intensity and angle.
- **Shader-Based Animation:** These original planning points were maintained. Vertex shaders were used for subtle, continuous animations to add life to static models.

3 Core Technical Implementation

3.1 Basic Scene Setup and Object Loading

The foundational scene environment was established using C++ and the OpenGL API. Standard libraries were used for initialization and context management: GLFW for window creation and input handling, and GLAD for loading OpenGL function pointers. The use of these standardized libraries ensured the project was platform-independent and adhered to modern OpenGL core profile requirements.

Code Snippet 3.1: OpenGL Initialization and Context Setup

```
// 1. Initializing GLFW
if(!glfwInit()){
    std::cerr << "Failed to initialize GLFW" << std::endl;
    return -1;
}
// Setting up OpenGL version
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 2);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
// ... creating window and loading GLAD
```

- **Model Loading:** External 3D model(the castle, statue, and wizards) were crucial to the scene's aesthetic. These models were loaded using the assimp library. Assimp was responsible for parsing various file formats(obj in my project), extracting necessary data like vertex coordinates, normals, and texture coordinates, and ensuring they were correctly translated into the simulation environment.
- **Data Structure:** All loaded geometry was effectively managed using vertex array objects(VAO) and vertex buffer objects(VBO) to transfer data to GPU memory once and render it repeatedly in the main loop.
- **Terrain and Environment:** The base of the floating island was implemented as a large, indexed mesh, providing the central courtyard surface. The scene was enveloped by a large-scale skybox model which was provided the visual backdrop for the magical blue sky and the vast space below the floating island.

Code Snippet 3.2: Model Loading in Main

```
Model castelModel("Wizard Academy/assets/Model/castle/de_haar_castle.obj");
Model skyboxModel("Wizard Academy/assets/Model/skybox/free_skybox_fantasy_lands_cloudy_space.
// ... and other model definitions
```

3.2 User Camera Control

User interaction was implemented via a customer Camera class allowing fluid exploration of the environment. This provided the scene with an interactive, first-person perspective essential for viewing the dynamic effects.

- **Input Handling:** Standard keyboard input (W, A, S, D) was mapped for forward/backward movement while the mouse controlled the camera's pitch(up/down) and yaw(left/right) rotation. The *processInput* function handled continuous input polling during the render loop to ensure smooth movement, and mouse callbacks managed the camera rotation based on cursor position changes.
- **View Matrix:** The core of the camera control involves calculating the View Matrix. This matrix is continuously updated using the look-at function, defining the camera's current position, it's target direction, and it's up orientation. This matrix is then passed to the vertex shaders for every object render, translating the world into the camera's perspective.

Code Snippet 3.3: Input Processing for Camera Movement

```
void processInput(GLFWwindow *window)
{
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        camera.ProcessKeyboard(FORWARD, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        camera.ProcessKeyboard(BACKWARD, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        camera.ProcessKeyboard(LEFT, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
```

```

        camera.ProcessKeyboard(RIGHT, deltaTime);
    // ... Additional UP/DOWN controls
}

```

3.3 Hierarchical Animation

The requirement for hierarchical animation was met by designing and implementing a custom, time-based, linked particle system to visually represent the flow of magical energy being channeled by the wizards. This approach was chosen to create a more dynamic and thematic visual and thematic experience, placing the animation focus directly on the magical power.

- **Hierarchal Structure:** The animation establishes a conceptual parent-child relationship that is realized through matrix transformations:
- **Parent Node(Root):** The wizard's position which acts as the dynamic base for the channeling effect. This position is calculated using the wizard's world matrix, meaning the entire swirling effect correctly follows the wizard's movement, including any vertical translation which will depict the floating wizard.
- **Child Nodes(Element):** The individual, animated magical particles generated around the hand.
- **Animation Logic:** The motion of each child particle is complex, time-dependent, and precisely calculated relative to the parent's coordinates. The particles execute a multi-faced motion to simulate swirling energy:
 - **Linked Coordinates:** The particle positions are calculated using the parent matrix's spatial orientation vectors to ensure the entire swirling effect is correctly oriented in the world, providing the structural linkage.
 - **Combined Motion:** The movement is composed of two primary time-dependent functions:
 - * **Sinusoidal Oscillation:** A sin function of time creates a vertical breathing or bobbing motion for the entire energy field which is simulating the channeling of magic.
 - * **Circular Orbit:** A second time-dependent function causes the particles to simultaneously execute a circular orbit around the wizard's body, creating the vortex effect.
- **Fulfillment of Requirement:** This particle system design successfully demonstrates the core principles of hierarchical animation, complex motion that is calculated per frame and applied relative to a parent coordinate system, thereby fulfilling the requirement for a complex, meaningful, time-based animation.

Code Snippet 3.4: Particle Position Calculation (Child Node Transformation)

```

// Inside DrawSimpleParticleSpell
// Extract parent position and rotation from the wizard's model matrix
glm::vec3 parentPos = glm::vec3(parentModel[3]);
glm::mat3 parentRot = glm::mat3(...);
glm::vec3 right = parentRot[0];

```



```

glm::vec3 up = parentRot[1];
glm::vec3 forward = parentRot[2];

for (int i = 0; i < numParticles; ++i) {
    float angle = time * orbitSpeed + i * phasePerParticle;
    float verticalOffset = verticalAmp * sin(time * verticalSpeed + i * 0.3f);

    // Complex motion relative to parent's orientation vectors
    glm::vec3 offset =
        right * (radius * cos(angle)) + // Circular Orbit
        forward * (radius * sin(angle)) +
        up * verticalOffset;           // Vertical Sinusoidal Oscillation

    glm::vec3 pos = parentPos + offset; // Final position is parentPos + offset
    // ...
}

```

4 Lighting and Shading Implementation

4.1 Dynamic time-of-Day System (Advanced Feature)

A core technical achievement of the project is the implementation of a full, procedural time-of-cycle(Daylight -> Sunset -> Night -> Daylight). This system dynamically controls the scene's entire lighting environment to communicate the passage of time within the creative constraint of the always-blue magical sky.

- **Time Calculation and Control:** The cycle is driven by the continuous run time *glfwGetTime*. A central *timeOfDay* variable is generated using the formula $\sin(\omega t \times 0.5 + 0.5)$, ensuring a smooth, predictable oscillation between 0.0 and 1.0. This value dictates the current point in the 24-hour cycle.
- **Color Interpolation:** The light's color is calculated dynamically using linear interpolation *glm::mix* between four defined phases: Night, Sunrise, Day and Sunset. this ensures the primary light source smoothly transitions its color temperature from cool blue/gray to warm orange/red, and finally to bright white/yellow.
- **Light Direction and Shadow Angle:** The directional light's angle *lightDir* is simultaneously updated based on trigonometric functions of the current frame time. This causes the light to sweep across the environment, making the shadows rotate across the courtyard. The changing shadow angle, combined with the shifting color temperature, is the primary visual cue used to communicate the time of day to the user.
- **Magical Sky Constraint:** To maintain the magical realm's aesthetic the clear color is custom-calculated to remain a shade of blue. Instead of allowing the sky to naturally transition to black/red, the system mixes the base magical blue color with a dark factor based on *timeOfDay*, fulfilling the creative mandate by using shadow and material color to show time, rather than the sky itself.

Code Snippet 4.1: Time and Light Color Interpolation Logic

```

float currentFrame = static_cast<float>(glfwGetTime());
// timeOfDay oscillates between 0.0 and 1.0
float timeOfDay = sin(currentFrame * 0.05f) * 0.5f + 0.5f;

glm::vec3 dayColor = glm::vec3(1.3f, 1.25f, 1.1f);
glm::vec3 nightColor = glm::vec3(0.2f, 0.3f, 0.5f);
glm::vec3 sunsetColor = glm::vec3(1.5f, 0.6f, 0.3f);

glm::vec3 lightColor;
if (timeOfDay < 0.3f) {
    // Night to sunrise mix
    float t = timeOfDay / 0.3f;
    lightColor = glm::mix(nightColor, sunsetColor, t);
} else if (timeOfDay < 0.5f) {
    // Sunrise to day mix
    // ... (logic continues for all phases)
}
// Sky color constraint implemented here for glClearColor
glm::vec3 skyMagicColor = glm::vec3(0.529f, 0.808f, 0.922f);
// ... mix logic applied to skyMagicColor
glClearColor(skyColor.r, skyColor.g, skyColor.b, 1.0f);

```

Code Snippet 4.2: Light Direction Calculation

```

float cycleSpeed = 0.1f;
float sunAngle = currentFrame * cycleSpeed;
glm::vec3 lightDir = glm::normalize(glm::vec3(
    cos(sunAngle) * 0.6f,    // X component
    -sin(sunAngle) * 0.8f,  // Y component (height/altitude)
    -0.3f
));
defaultShader.setVec3("lightDirection", lightDir);
defaultShader.setVec3("lightColor", lightColor);

```

4.2 Phong illumination and Light Sources

The scene's realism and visual polish are achieved through the implementation of the Phong Illumination Model, calculated per fragment(pixel) within the fragment shader. This model accurately simulates how light interacts with the materials of the models.

- **Model Components:** The Phong calculates three components of light interaction:
 - **Ambient:** Represents generalized scattered light.
 - **Diffuse:** Represents light scattered evenly across the surface.
 - **Specular:** Represents highlights that depends on the viewer's position.
 The overall light intensity for any given fragment is the sum of these three components.

- **Shader Configuration:** The model relies on passing essential uniforms to the shader, including the viewer's position (*viewPos*), the object's smoothness/shininess (*shininess*), and the dynamic lighting properties (*lightDirection* and *lightcolor*).
- **Normal Transformation:** Critically, the surface normals are transformed into world space using the appropriate Normal Matrix. This ensures the diffuse and specular calculations remain accurate even when models are scaled or rotated.

noindent

Code Snippet 4.3: Phong Uniform Setup

```
// Setup uniforms for Phong calculation in the fragment shader
defaultShader.setMat4("projection", projection);
defaultShader.setMat4("view", view);

defaultShader.setVec3("lightDirection", lightDir);
defaultShader.setVec3("lightColor", lightColor);
defaultShader.setVec3("viewPos", camera.Position);
defaultShader.setFloat("shininess", 16.0f);           // Specular Exponent (n)
```

4.3 Texture Mapping

Texture mapping was utilized across all imported 3D models and the terrain to provide realistic surface detail and color, enhancing the animated fantasy aesthetic of the academy.

- *Texture Loading:* Textures associated with the imported models were loaded through the Assimp framework. The texture data was converted into OpenGL texture objects, enabling GPU access for shading.
- *Texture Types and Application:*
 - *Diffuse Maps:* Applied to the materials to provide the base color and fine-grain surface detail.
 - *Specular Reflection (Uniform):* Since dedicated specular map textures were not used, the specular reflection property for all materials was treated as uniform constant within the fragment shader. The intensity and spread of the highlight are governed by the shininess uniform.
- *UV Coordinates:* The models imported using assimp provided necessary UV coordinates per vertex, which were passed through the vertex shader and interpolated to the fragment shader. The shader then uses these coordinates to sample the texture, accurately painting the surface of the 3D geometry.

5 Advance Rendering and Final Polish

5.1 Dynamic Magical Effects

The magical effects of the spell casting wizards are the visual centerpiece of the simulation. Achieving the vivid, luminous quality of the spells required implementing a custom particle system combined with specialized OpenGL blending techniques.

- **Particle Geometry:** The spells are not rendered using complex 3D geometry but rather highly efficient, textured 2D quads *SimpleParticle*. The geometry is rendered using the billboard technique, where the shader uses the camera's *cameraRight* and *cameraUp* vectors to dynamically orient the quads to perpetually face the viewer. This creates the illusion of a volumetric, spherical particle cloud while keeping the geometry cost extremely low.
- **Hierarchical Motion:** As detailed in Section 3.3, each particle's position is calculated in the *DrawSimpleParticleSpell* function relative to the wizard. The motion logic combines two cosine wave functions to produce a complex path of circular orbit and vertical oscillation, simulating the turbulent, channeled energy.
- **Additive Blending:** The appearance is achieved by setting the global blending function to Additive Blending before rendering the spells. This powerful technique physically adds the light output of the particles to the existing color buffer, making the spells look like actual light sources that retain their brilliance even in the bright daylight/sunset environment.

Code Snippet 5.1: Additive Blending Activation

```
// Enable additive blending for particles
glDepthMask(GL_FALSE); // Disable writing to the depth buffer
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE); // SOURCE_ALPHA, ONE = Additive Blending

particleShader.use();
// ... Draw Particles (using the custom particle system)

// Reset blending state
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_TRUE);
glDisable(GL_BLEND);
```

Code Snippet 5.2: Particle Billboard Setup

```
// Inside particle drawing loop (DrawSimpleParticleSpell):
shader.setVec3("cameraRight", cameraRight); // Passed to the vertex shader
shader.setVec3("cameraUp", cameraUp);       // for billboard calculation
// ...
particle.Draw(); // Draw the 2D quad geometry
```

5.2 Complex Time-Based Transformation: Floating Wizard

A high-impact animation was implemented on Wizard 4 to visually signify the successful channeling and release of potent magical energy.

- **Feature Goal:** To have one wizard smoothly rise and float off the ground after fully channeling the spell, adding a dramatic focal point to the scene and illustrating the powerful effect of the magic.

- **Implementation:** This dramatic effect is achieved via a time-dependent modification of the wizard's translation matrix. The model's Y-translation component is gradually increased using the continuous run time.
- **Visual Impact:** This linear ascent demonstrates the intentional use of a complex time-based translation transformation to enhance the scene's narrative and visual hierarchy, shoeing the mastery of the spell.

Code Snippet 5.3: Floating Transformation (Conceptual Implementation)

```
// NOTE: This logic needs to be integrated into main.cpp for Wizard 4's matrix.
float rise_time = currentFrame - 10.0f; // Assumes rising starts at 10 seconds
float vertical_offset = 0.0f;

if (currentFrame > 10.0f) {
    // Gradual rise (e.g., linear rise after the delay)
    vertical_offset = rise_time * 0.5f;
}

glm::mat4 wizard4Matrix = glm::mat4(1.0f);
wizard4Matrix = glm::translate(wizard4Matrix, glm::vec3(-59.0f, 10.0f + vertical_offset, -13.0f));
// ... rest of rotation and scale
```

5.2.1 Overall Aesthetic and Immersive Quality:

The final aesthetic integrates the technical components to create a cohesive and immersive environment:

- **Visual Cohesion:** The use of the dynamic time-of-day system ensures all lighting remains consistent and harmonized, from the cool tones of night to the warm tones of sunset, all while maintaining the unique magical blue sky constraint.
- **Stylized Fidelity:** The combination of Phong shading on the architectural models and the intense Additive Blending on the magical spells successfully captures the desired animated fantasy style, creating a vibrant and dynamic visual experience.
- **Fluid Interaction:** The continuous, complex motion of the particle systems and the interactive camera control allow the user to explore a living, dynamic ecosystem, successfully fulfilling the project's goal of creating an immersive simulation.

6 Conclusion and Future work

6.1 Project Summary and Achieved Objectives

The project successfully delivered a complex Fantasy Ecosystem Simulation, The Wizard's Academy, utilizing a shader-based OpenGL framework. The final scene fully realized the thematic goal of presenting a dynamic magical training environment, even after strategically revising the scope to include a floating island and a dynamic time-of-day cycle.

All core requirements across the three deliverable were met:

- **Core Scene and Interactivity:** Successful loading of all 3D geometry via assimp, implementation of fluid camera controls, and fulfillment of the hierarchal animation requirement through the custom, time-based particle system.

- **Rendering and Advanced Features:** Implement of the Phong Illumination Model, application of texture mapping for visual detail, and the integration of multiple advanced features:
 1. The Dynamic Time-of-Day System
 2. The Custom Particle System for spell efforts
 3. Additive Blending for true light emission
 4. The Time-Based Ascending Transformation for the climax animation of the floating wizard.

6.2 Challenges and Learning Outcomes

The development process presented several critical technical challenges:

- **Dynamic Light Integration:** A primary challenge was ensuring the Additive Blending on the luminous spells remained impactful and accurate across the entire dynamic lighting cycle. The required careful calibration of the primary directional light's intensity versus the particle color output.
- **Hierarchical Motion:** Implementing the complex, linked motion of the particle system relative to the wizard's based calculations to achieve the visual effect of channeled energy without relying on traditional bone animation.

The project served as a comprehensive learning exercise, particular in the use of uniform manipulation in shaders to control dynamic effects and the creative application of blending techniques to achieve specific visual goals.

7 References and Appendices

7.1 Source Code Repository

The complete source code for the Wizard's Academy project (C++ and GLSL shaders) is hosted and managed in this repository:

- **GitHub Repository:** <https://github.com/dik17sha/Wizard-s-Academy>

7.2 Technical References

- **OpenGL Shading Language (GLSL) Specification:** Khronos Group. *The OpenGL Shading Language, Version 4.60*. Available from the Khronos OpenGL Registry. (Required for understanding shader syntax, built-in functions, and uniform management.)
- **Model Loading (Assimp):** Open Asset Import Library Development Team. *ASSIMP Documentation*. Available on GitHub/Official Assimp Website. (Used for parsing OBJ file formats and extracting model data.)
- **Mathematics Library (GLM):** G-Truc Creation. *OpenGL Mathematics (GLM)*. Available on the Official GLM Website. (Used for all matrix transformations, vector operations, and trigonometric functions in C++.)
- **Learning Resource (Tutorials):** Learning OpenGL Tutorials. *LearnOpenGL: The comprehensive OpenGL resource*. Available at [<https://learnopengl.com/>] (<https://learnopengl.com/>). (Used as a general guide for modern OpenGL setup, Phong lighting implementation, and advanced blending concepts.)

- **Lighting and Blending Specifications:** Khronos Group. *OpenGL Reference Pages: glBlendFunc and glDepthFunc*. Available from the Khronos OpenGL Registry. (Specifically referenced for implementing the ****Additive Blending**** used for magical effects and defining the ****Phong Illumination Model****.)

7.3 Asset References

- **Wizard Statue (Focal Point):** "PBR Stylized Statue - Mage / Wizard" by **Ignas**. <https://sketchfab.com/3d-models/pbr-stylized-statue-mage-wizard-f0293fab679e44d1bfa38798e06db307>
- **Head Wizard Model (Dumbledore):** "Dumbledore" by **zack_graham**. <https://sketchfab.com/3d-models/dumbledore-bcb0ca50e18544d299e496521abed7ef>
- **Student Model (Draco Malfoy):** "Draco Malfoy (CoS, PS2)" by **Chronis**. <https://sketchfab.com/3d-models/draco-malfoy-cos-ps2-6139011998a044409c0ca310ef1e8d79>
- **Student Model (Harry Potter):** "Harry Potter (CoS, GameCube/Xbox)" by **Chronis**. <https://sketchfab.com/3d-models/harry-potter-cos-gamecubexbox-7f3edb59fadd4aada5eb8c6f2b06ca6d>
- **Student Model (Ron Weasley):** "Ron Weasley" by **Chronis**. <https://sketchfab.com/3d-models/ron-weasley-85792d588da2432bbd64242af2670b96>
- **Student Model (Hermione Granger):** "Hermione Granger (CoS, GameCube/Xbox)" by **Chronis**. <https://sketchfab.com/3d-models/hermione-granger-cos-gamecubexbox-36a7f1dbfb184c>
- **Spell Particle Geometry:** "uttm_glow" by **AASK STUDIO**. <https://sketchfab.com/3d-models/uttm-glow-5e542fb62460448fbd922050296bcdea>
- **Skybox/Atmosphere:** "Skybox 002" by [stylized skybox], used to frame the environment and simulate the clouds beneath the floating island. <https://sketchfab.com/3d-models/skybox-002-294f698e0fef40b0ba6ad261df757518>

7.4 Appendix A: Demonstration Video

The complete functionality and final aesthetic of the simulation are demonstrated in the required video clip: https://youtu.be/rMm-r9_Ty1k